

# Импорт данных : : ШПАРГАЛКА

В R **tidyverse** построена вокруг **опрятных данных**, хранящихся в **tibble** (развитие data frame).



На лицевой стороне описано, как считывать текстовые файлы в R с помощью **readr**.



На обратной стороне описано, как создавать tibble с **tibble** и опрятные данные с **tidyr**.

## ДРУГИЕ ТИПЫ ДАННЫХ

Используйте следующие пакеты для импорта файлов другого типа

- **haven** - файлы SPSS, Stata и SAS
- **readxl** - excel файлы (.xls и .xlsx)
- **DBI** - базы данных
- **jsonlite** - json
- **xml2** - XML
- **httr** - Web API
- **rvest** - HTML (данные из Интернета)

## Сохранение данных

Сохраняйте **x**, объект R, в **path**, путь к файлу, с помощью:

### Файл с разделителем запятой

**write\_csv**(x, path, na = "NA",  
append = FALSE, col\_names = !append)

### Файл с произвольным разделителем

**write\_delim**(x, path, delim = " ", na = "NA",  
append = FALSE, col\_names = !append)

### CSV для excel

**write\_excel\_csv**(x, path, na = "NA",  
append = FALSE, col\_names = !append)

### Строка в файл

**write\_file**(x, path, append = FALSE)

### Строковый вектор в файл, один элемент на строку

**write\_lines**(x, path, na = "NA",  
append = FALSE)

### Объект в файл RDS

**write\_rds**(x, path, compress = c("none", "gz",  
"bz2", "xz"), ...)

### Файл с разделителем табуляцией

**write\_tsv**(x, path, na = "NA",  
append = FALSE, col\_names = !append)

## Считывание табличных данных

Эти функции имеют общие аргументы:

**read\_\***(file, col\_names = TRUE, col\_types = NULL, locale = default\_locale(), na = c("", "NA"),  
quoted\_na = TRUE, comment = "", trim\_ws = TRUE, skip = 0, n\_max = Inf, guess\_max =  
min(1000, n\_max), progress = interactive())

a,b,c  
1,2,3  
4,5,NA

| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

### Файлы с разделителем запятой

**read\_csv**("file.csv")

Для создания file.csv выполните:  
**write\_file**(x = "a,b,c\n1,2,3\n4,5,NA", path = "file.csv")

a;b;c  
1;2;3  
4;5;NA

| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

### Файлы с разделителем точкой с запятой

**read\_csv2**("file2.csv")

**write\_file**(x = "a;b;c\n1;2;3\n4;5;NA", path = "file2.csv")

alblc  
1|2|3  
4|5|NA

| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

### Файлы с произвольным разделителем

**read\_delim**("file.txt", delim = "|")

**write\_file**(x = "alblc\n1|2|3\n4|5|NA", path = "file.txt")

a b c  
1 2 3  
4 5 NA

| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

### Файлы с фиксированной шириной

**read\_fwf**("file.fwf", col\_positions = c(1, 3, 5))

**write\_file**(x = "a b c\n1 2 3\n4 5 NA", path = "file.fwf")

### Файлы с разделителем табуляцией

**read\_tsv**("file.tsv") Также **read\_table**()

**write\_file**(x = "a\tb\tc\n1\t2\t3\n4\t5\tNA", path = "file.tsv")

## ПОЛЕЗНЫЕ АРГУМЕНТЫ

a,b,c  
1,2,3  
4,5,NA

### Файл для примеров

**write\_file**("a,b,c\n1,2,3\n4,5,NA", "file.csv")  
f <- "file.csv"

| 1 | 2 | 3  |
|---|---|----|
| 4 | 5 | NA |

### Пропуск строк

**read\_csv**(f, **skip** = 1)

| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

### Без заголовка

**read\_csv**(f, **col\_names** = FALSE)

| A | B | C |
|---|---|---|
| 1 | 2 | 3 |

### Считывание части

**read\_csv**(f, **n\_max** = 1)

| x | y | z  |
|---|---|----|
| A | B | C  |
| 1 | 2 | 3  |
| 4 | 5 | NA |

### С указанием заголовка

**read\_csv**(f, **col\_names** = c("x", "y", "z"))

| A  | B | C  |
|----|---|----|
| NA | 2 | 3  |
| 4  | 5 | NA |

### Пропущенные значения

**read\_csv**(f, **na** = c("1", "."))

## Считывание нетабличных данных

### Чтение файла в одну строку

**read\_file**(file, locale = default\_locale())

### Чтение строк файла в отдельные строки

**read\_lines**(file, skip = 0, n\_max = -1L,  
na = character(), locale = default\_locale(),  
progress = interactive())

### Чтение логовых файлов Apache

**read\_log**(file, col\_names = FALSE, col\_types = NULL, skip = 0, n\_max = -1, progress = interactive())

### Чтение файла в raw вектор

**read\_file\_raw**(file)

### Чтение строк файла в raw векторы

**read\_lines\_raw**(file, skip = 0, n\_max = -1L,  
progress = interactive())



## Типы данных

Функции **readr** угадывают типы столбцов и преобразуют их, если считают уместным (но НИКОГДА строки в факторы автоматически).  
Сообщение описывает типы столбцов в результате.

```
## Parsed with column specification:
## cols(
##   age = col_integer(),
##   sex = col_character(),
##   earn = col_double()
## )
```

age -  
целое  
число

sex -  
строка

earn - число double

1. Используйте **problems()** для диагностики проблем

**x <- read\_csv("file.csv"); problems(x)**

2. Используйте **col\_** функцию для разбора

- **col\_guess()** - по умолчанию
- **col\_character()**
- **col\_double()**, **col\_euro\_double()**
- **col\_datetime**(format = "") Также **col\_date**(format = ""), **col\_time**(format = "")
- **col\_factor**(levels, ordered = FALSE)
- **col\_integer()**
- **col\_logical()**
- **col\_number()**, **col\_numeric()**
- **col\_skip()**

**x <- read\_csv("file.csv", col\_types = cols(  
A = col\_double(),  
B = col\_logical(),  
C = col\_factor()))**

3. По-другому: считывайте как символьные векторы и разбирайте с **parse\_** функцией

- **parse\_guess()**
- **parse\_character()**
- **parse\_datetime()** Также **parse\_date()** и **parse\_time()**
- **parse\_double()**
- **parse\_factor()**
- **parse\_integer()**
- **parse\_logical()**
- **parse\_number()**

**x\$A <- parse\_number(x\$A)**

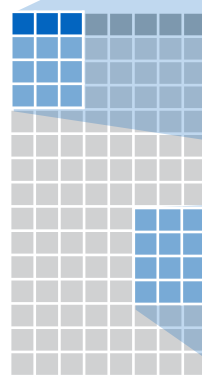


# Tibble - развитие data frame

Пакет **tibble** вводит новый S3 класс для хранения табличных данных, **tibble**. **Tibble** наследует класс **data frame**, но улучшает три направления:



- **Выбор элементов** - [ всегда возвращает новый **tibble**, [[ и \$ - вектор.
- **Нет частичного соответствия** - Вы должны использовать полные имена столбцов при выборе элементов.
- **Отображение** - При печати **tibble**, R выводит краткий вид данных, уместающийся на один экран.



Большая таблица

```
# A tibble: 234 x 6
  manufacturer <chr> model <chr> displ <dbl>
1 audi a4 1.8 1.8
2 audi a4 2.0 2.0
3 audi a4 2.0 2.0
4 audi a4 2.0 2.0
5 audi a4 2.0 2.0
6 audi a4 2.0 2.0
7 audi a4 2.0 2.0
8 audi a4 2.0 2.0
9 audi a4 2.0 2.0
10 audi a4 2.0 2.0
... with 224 more rows, and 3 more variables: year <int>, cyl <int>, trans <chr>
```

вид tibble

```
156 1999 6 auto(l4)
157 1999 6 auto(l4)
158 2008 6 auto(l4)
159 2008 8 auto(s4)
160 1999 4 manual(m5)
161 1999 4 auto(l4)
162 2008 4 manual(m5)
163 2008 4 manual(m5)
164 2008 4 auto(l4)
165 2008 4 auto(l4)
166 1999 4 auto(l4)
#> reached getOption("max.print")
#> omitted 68 rows ]
```

вид data frame

- Управление видом по умолчанию опциями:  
**options(tibble.print\_max = n, tibble.print\_min = m, tibble.width = Inf)**
- Просмотр полных данных: **View()** или **glimpse()**
- Возврат к data frame: **as.data.frame()**

## СОЗДАНИЕ TIBBLE ДВУМЯ СПОСОБАМИ

**tibble(...)**  
Создает по столбцам.

**tibble(x = 1:3, y = c("a", "b", "c"))**

Оба создают этот tibble

**tribble(...)**  
Создает по строкам.

**tribble(~x, ~y,  
1, "a",  
2, "b",  
3, "c")**

```
A tibble: 3 x 2
  x     y
<int> <chr>
1     1 a
2     2 b
3     3 c
```

**as\_tibble(x, ...)**

Конвертирует data frame в tibble.

**enframe(x, name = "name", value = "value")**

Конверт. именованный вектор в tibble

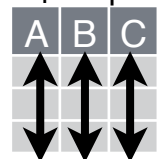
**is\_tibble(x)** Проверяет, явл. ли x tibble.



# Опрятные данные с tidyr

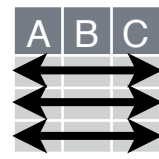
**Опрятные данные** - это способ организации табличных данных. Это устанавливает структуру данных, согласованную между пакетами.

Таблица опрятна, если:



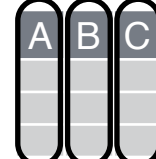
Каждая переменная - столбец

&

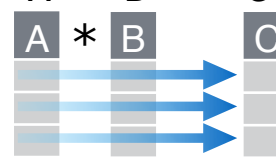


Каждое наблюдение - строка

В опрятных данных:



Легко обращаться к переменным, как к векторам



Сохраняем наблюдения при векторизованных операциях

## Форматирование данных

Изменение расположения элементов в таблице

Используйте **gather()** и **spread()** для реорганизации элементов таблицы.

**gather(data, key, value, ..., na.rm = FALSE, convert = FALSE, factor\_key = FALSE)**  
**spread(data, key, value, fill = NA, convert = FALSE, drop = TRUE, sep = NULL)**

Перемещает имена столбцов в столбец **key**, собирая значения столбцов в общий столбец **value**.

table4a

| country | 1999 | 2000 |
|---------|------|------|
| A       | 0.7K | 2K   |
| B       | 37K  | 80K  |
| C       | 212K | 213K |



| country | year | cases |
|---------|------|-------|
| A       | 1999 | 0.7K  |
| B       | 1999 | 37K   |
| C       | 1999 | 212K  |
| A       | 2000 | 2K    |
| B       | 2000 | 80K   |
| C       | 2000 | 213K  |

key value

**gather(table4a, `1999`, `2000`,  
key = "year", value = "cases")**

table2

| country | year | type  | count |
|---------|------|-------|-------|
| A       | 1999 | cases | 0.7K  |
| A       | 1999 | pop   | 19M   |
| A       | 2000 | cases | 2K    |
| A       | 2000 | pop   | 20M   |
| B       | 1999 | cases | 37K   |
| B       | 1999 | pop   | 172M  |
| B       | 2000 | cases | 80K   |
| B       | 2000 | pop   | 174M  |
| C       | 1999 | cases | 212K  |
| C       | 1999 | pop   | 1T    |
| C       | 2000 | cases | 213K  |
| C       | 2000 | pop   | 1T    |

key value

**spread(table2, type, count)**

| country | year | cases | pop  |
|---------|------|-------|------|
| A       | 1999 | 0.7K  | 19M  |
| A       | 2000 | 2K    | 20M  |
| B       | 1999 | 37K   | 172M |
| B       | 2000 | 80K   | 174M |
| C       | 1999 | 212K  | 1T   |
| C       | 2000 | 213K  | 1T   |

## Работа с пропущенными значениями

**drop\_na(data, ...)**

Убирает строки с NA в столбцах из ...

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | NA |
| C  | NA |
| D  | 3  |
| E  | NA |

**drop\_na(x, x2)**

**fill(data, ..., .direction = c("down", "up"))**

Заменяет NA в столбцах из ... крайними не-NA значениями.

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | 1  |
| C  | 1  |
| D  | 3  |
| E  | 3  |

**fill(x, x2)**

**replace\_na(data, replace = list(), ...)**

Заменяет NA по столбцам.

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | 2  |
| C  | 2  |
| D  | 3  |
| E  | 2  |

**replace\_na(x, list(x2 = 2))**

## Расширение таблиц - быстрое создание таблиц с комб-ми значений

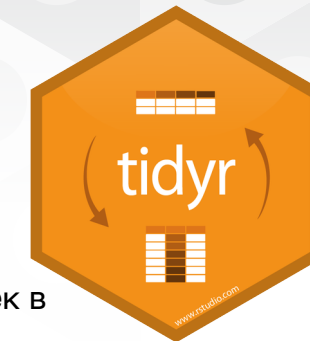
**complete(data, ..., fill = list())**

Добавляет к данным недостающие комбинации значений переменных из ...  
**complete(mtcars, cyl, gear, carb)**

**expand(data, ...)**

Создает новый tibble со всеми комбинациями значений переменных из ...  
**expand(mtcars, cyl, gear, carb)**

# Деление ячеек



Функции для разделения или комбинирования ячеек в отдельные значения.

**separate(data, col, into, sep = "[^:alnum:]", remove = TRUE, convert = FALSE, extra = "warn", fill = "warn", ...)**

Разделяет ячейки столбца по отдельным столбцам.

table3

| country | year | rate     |
|---------|------|----------|
| A       | 1999 | 0.7K/19M |
| A       | 2000 | 2K/20M   |
| B       | 1999 | 37K/172M |
| B       | 2000 | 80K/174M |
| C       | 1999 | 212K/1T  |
| C       | 2000 | 213K/1T  |



| country | year | cases | pop |
|---------|------|-------|-----|
| A       | 1999 | 0.7K  | 19M |
| A       | 2000 | 2K    | 20M |
| B       | 1999 | 37K   | 172 |
| B       | 2000 | 80K   | 174 |
| C       | 1999 | 212K  | 1T  |
| C       | 2000 | 213K  | 1T  |

**separate(table3, rate,  
into = c("cases", "pop"))**

**separate\_rows(data, ..., sep = "[^:alnum:]", convert = FALSE)**

Разделяет ячейки столбца по отдельным строкам. Также **separate\_rows()**.

table3

| country | year | rate     |
|---------|------|----------|
| A       | 1999 | 0.7K/19M |
| A       | 2000 | 2K/20M   |
| B       | 1999 | 37K/172M |
| B       | 2000 | 80K/174M |
| C       | 1999 | 212K/1T  |
| C       | 2000 | 213K/1T  |



| country | year | rate |
|---------|------|------|
| A       | 1999 | 0.7K |
| A       | 1999 | 19M  |
| A       | 2000 | 2K   |
| A       | 2000 | 20M  |
| B       | 1999 | 37K  |
| B       | 1999 | 172M |
| B       | 2000 | 80K  |
| B       | 2000 | 174M |
| C       | 1999 | 212K |
| C       | 1999 | 1T   |
| C       | 2000 | 213K |
| C       | 2000 | 1T   |

**separate\_rows(table3, rate)**

**unite(data, col, ..., sep = "\_", remove = TRUE)**

Объединяет ячейки нескольких столбцов в один столбец.

table5

| country | century | year |
|---------|---------|------|
| Afghan  | 19      | 99   |
| Afghan  | 20      | 0    |
| Brazil  | 19      | 99   |
| Brazil  | 20      | 0    |
| China   | 19      | 99   |
| China   | 20      | 0    |



| country | year |
|---------|------|
| Afghan  | 1999 |
| Afghan  | 2000 |
| Brazil  | 1999 |
| Brazil  | 2000 |
| China   | 1999 |
| China   | 2000 |

**unite(table5, century, year,  
col = "year", sep = "")**